815. Bus Routes

```
#include <stdlib.h>
#include <string.h>

typedef struct {
    int stop;
    int routeIdx;
} StopRoutePair;

int comparePairs(const void* a, const void* b) {
    StopRoutePair* p1 = (StopRoutePair*)a;
    StopRoutePair* p2 = (StopRoutePair*)b;
    if (p1->stop != p2->stop) return p1->stop - p2->stop;
    return p1->routeIdx - p2->routeIdx;
}

int numBusesToDestination(int** routes, int routesSize, int* routesColSize, int source, int target) {
    if (source == target) return 0;

    int totalPairs = 0;
    for (int i = 0; i < routesSize; i++) {
        totalPairs += routesColSize[i];
    }

    StopRoutePair* pairs = (StopRoutePair*)malloc(totalPairs * sizeof(StopRoutePair));
    int pairCount = 0;
    for (int i = 0; i < routesSize; i++) {
        for (int j = 0; j < routesColSize[i]; j++) {
            pairs[pairCount].stop = routes[i][j];
            pairs[pairCount].routeIdx = i;
            pairCount++;
        }
    }

    qsort(pairs, totalPairs, sizeof(StopRoutePair), comparePairs);

    int* stopHead = (int*)malloc((totalPairs + 1) * sizeof(int));
    for (int i = 0; i <= totalPairs; i++) stopHead[i] = -1;

    int uniqueStops = 0;
    for (int i = 0; i < totalPairs; i++) {
        if (i == 0 || pairs[i].stop != pairs[i - 1].stop) {
            stopHead[uniqueStops] = i;
            uniqueStops++;
        }
    }
    stopHead[uniqueStops] = totalPairs;

    int findStopIdx(int stopVal) {
        int low = 0, high = uniqueStops - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            int currentStop = pairs[stopHead[mid]].stop;
            if (currentStop == stopVal) return mid;
            if (currentStop < stopVal) low = mid + 1;
            else high = mid - 1;
```

```
        }
        return -1;
    }

    int* queue = (int*)malloc(routesSize * sizeof(int));
    int* routeBuses = (int*)malloc(routesSize * sizeof(int));
    int head = 0, tail = 0;

    int* visitedRoutes = (int*)calloc(routesSize, sizeof(int));

    int sourceIdx = findStopIdx(source);
    if (sourceIdx == -1) {
        free(pairs); free(stopHead); free(queue); free(routeBuses); free(visitedRoutes);
        return -1;
    }

    for (int i = stopHead[sourceIdx]; i < stopHead[sourceIdx + 1]; i++) {
        int r = pairs[i].routeIdx;
        visitedRoutes[r] = 1;
        queue[tail] = r;
        routeBuses[tail] = 1;
        tail++;
    }

    while (head < tail) {
        int currRoute = queue[head];
        int busesTaken = routeBuses[head];
        head++;

        for (int i = 0; i < routesColSize[currRoute]; i++) {
            int stop = routes[currRoute][i];
            if (stop == target) {
                free(pairs); free(stopHead); free(queue); free(routeBuses); free(visitedRoutes);
                return busesTaken;
            }

            int stopIdx = findStopIdx(stop);
            if (stopIdx != -1) {
                for (int j = stopHead[stopIdx]; j < stopHead[stopIdx + 1]; j++) {
                    int nextRoute = pairs[j].routeIdx;
                    if (!visitedRoutes[nextRoute]) {
                        visitedRoutes[nextRoute] = 1;
                        queue[tail] = nextRoute;
                        routeBuses[tail] = busesTaken + 1;
                        tail++;
                    }
                }
            }
        }
    }

    free(pairs); free(stopHead); free(queue); free(routeBuses); free(visitedRoutes);
    return -1;
}
```

743. Network Delay Time

```c
#include <stdlib.h>
#include <limits.h>

int networkDelayTime(int** times, int timesSize, int* timesColSize, int n, int k) {
    int* dist = (int*)malloc((n + 1) * sizeof(int));
    int* visited = (int*)calloc((n + 1), sizeof(int));

    for (int i = 1; i <= n; i++) {
        dist[i] = INT_MAX;
    }
    dist[k] = 0;

    for (int i = 0; i < n; i++) {
        int u = -1;
        int minDist = INT_MAX;

        for (int j = 1; j <= n; j++) {
            if (!visited[j] && dist[j] < minDist) {
                minDist = dist[j];
                u = j;
            }
        }

        if (u == -1) break;
        visited[u] = 1;

        for (int j = 0; j < timesSize; j++) {
            if (times[j][0] == u) {
                int v = times[j][1];
                int w = times[j][2];
                if (dist[u] != INT_MAX && dist[u] + w < dist[v]) {
                    dist[v] = dist[u] + w;
                }
            }
        }
    }

    int maxDelay = 0;
    for (int i = 1; i <= n; i++) {
        if (dist[i] == INT_MAX) {
            free(dist);
            free(visited);
            return -1;
        }
        if (dist[i] > maxDelay) {
            maxDelay = dist[i];
        }
    }

    free(dist);
    free(visited);
    return maxDelay;
}
```